Telematics Platform User Guide

Common Services

Document Information	
Filename	TP_F101U-00009_User_Guide_Common_Service
Status	Release
Revision Date	Jul 19, 2021
Version	03.00
Document ID	TP_F101U-00009

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Table of contents

1 Introduction 4
1.1 Notational Conventions 4
2 Setup 4
2.1 Tools/Equipment Needed 4
2.2 Setup Instructions 4
2.3 Setup Troubleshooting 4
3 Log & Trace 5
3.1 Logging: Application usage 5
3.1.1 Overview 5
3.1.1.1 Platform Content Overview 5
3.1.1.2 Product Expectations 5
3.1.1.3 Extending Platform Functionality 6
3.1.2 Setup 6
3.1.2.1 Tools/Equipment Needed 6
3.1.2.2 Setup Instructions 6
3.1.2.3 Setup Troubleshooting 6
3.1.3 Usage 6
3.1.3.1 Basic Instructions 6
3.1.3.2 Detailed Instructions 6
3.1.3.3 Testing/Error Reporting 11
3.2 Logging: Configuring and routing log generation 11
3.2.1 Overview 11
3.2.1.1 Platform Content Overview 11
3.2.1.2 Product Expectations 11
3.2.1.3 Extending Platform Functionality 11
3.2.2 Setup 12
3.2.2.1 Tools/Equipment Needed 12
3.2.2.2 Setup Instructions 12
3.2.2.3 Setup Troubleshooting 12
3.2.3 Usage 12
3.2.3.1 Basic Instructions 12
3.2.3.2 Detailed Instructions 12
3.2.3.3 Testing/Error Reporting 14
4 Core dumps generation customization 14
4.1 How to configure core dump generation attributes 14
4.1.1 Overview 14
4.1.1.1 Platform Content Overview 14
4.1.1.2 Product Expectations 14
4.1.1.3 Extending Platform Functionality 14

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

4.1.2 Setup 14
4.1.2.1 Tools/Equipment Needed 14
4.1.2.2 Setup Instructions 14
4.1.2.3 Setup Troubleshooting 14
4.1.3 Usage 15
4.1.3.1 Basic Instructions 15
4.1.3.2 Detailed Instructions 15
4.1.3.3 Testing/Error Reporting 16
5 Known Issues 16
6 FAQs 16
7 Troubleshooting/Tips and Tricks 17
8 References 17
8.1 Abbreviations/Acronyms/Definitions 17
8.2 Platform Documentation 17
8.3 Website links 17
9 Revision History 18

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

1 Introduction

This document provides usage information for *common services*. A common service is a system service that is usable by components at all software layers (HAL components, Telematics Platform components, application components).

This document covers the following common services:

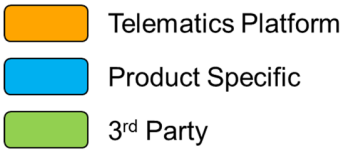
- Log & Trace, which provides capabilities for applications to log messages for evidence and debugging purposes.

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

2.1 Tools/Equipment Needed

Not Applicable

2.2 Setup Instructions

Not Applicable

2.3 Setup Troubleshooting

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3 Log & Trace

3.1 Logging: Application usage

3.1.1 Overview

3.1.1.1 Platform Content Overview

Telematics Platform provides a standard way to log information within separate applications using the syslog standard. Applications generate different logs at runtime that can be used to verify the correct functionality or debugging assistance when troubleshooting an error. The collection of all applications messages can be stored in different places: a log file inside the file system, sent over a network or a wired connection, or even RAM.

Messages are grouped by facilities already defined by the standard. These facilities represent subsystems generating messages and this simplifies their routing to specific locations. Telematics Platform has reserved four general purpose facilities (also referred internally as “domains”), while another four are available for application development teams. Next table shows the local facilities and its correspondent domain.

Syslog facility	Telematics Platform domain	Responsible project
`local0`	`TP_LOG_DOMAIN_PRODUCT`	Application development
`local1`	`TP_LOG_DOMAIN_PD_FLEX0`	Application development
`local2`	`TP_LOG_DOMAIN_PD_FLEX1`	Application development
`local3`	`TP_LOG_DOMAIN_PD_FLEX2`	Application development
`local4`	`TP_LOG_DOMAIN_PLF_RESERVE0`	Telematics Platform
`local5`	`TP_LOG_DOMAIN_PLF_RESERVE1`	Telematics Platform
`local6`	`TP_LOG_DOMAIN_PLF_RESERVE2`	Telematics Platform
`local7`	`TP_LOG_DOMAIN_PLATFORM`	Telematics Platform

Message logging includes the concept of message priority/severity levels, ranging from major system impact notices (e.g., fatal errors) to specific operation details (e.g., verbose information). Each log message is assigned a priority/severity level by the developer during code development.

Message log levels can be set on application basis, with increasing message traffic as log level is increased. Thus, using high log levels (e.g., verbose messaging) may impact performance including execution delays. As such, it is recommended that developers consider an appropriate message priority/severity level when defining log messages (i.e., don’t assign every message an “error” level).

Telematics Platform also provides support for stack traces for debugging purposes. For more information about this, please refer to section: “Miscellaneous interfaces for debugging” in TP_F101U-00023.

3.1.1.2 Product Expectations

The following expectations must be met by products (users of the platform) in order for this feature to work as designed:

1) The proper domain (syslog facility) is selected when opening a connection with the system logger with `tpLOG_open()`.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Note: `/data` partition should be available before calling `tpLOG_open()` API.

Note: Althoug a library can still log messages, these will appear in the context of the calling application. There's no need and is recommended to never call `tpLOG_open()` directly from a library.

2) Each application prioritizes the messages to be logged.

3) The proper log level is selected depending on the verbosity level required.

3.1.1.3 Extending Platform Functionality

Not applicable

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not applicable

3.1.2.2 Setup Instructions

Not applicable

3.1.2.3 Setup Troubleshooting

Not applicable

3.1.3 Usage

3.1.3.1 Basic Instructions

Not applicable

3.1.3.2 Detailed Instructions

References to Log & Trace types can be found in this section, for more details on these APIs and types see the Doxygen links in the following table:

API/Type Description	Doxygen Path

Logging severity level.	`Modules`
	`|-> Log And Trace`
	`|-> Logging (LOG)`
	`|-> tpLOG_severityLevel_t`

API to open a connection with the system logger.	`Modules`
	`|-> Log And Trace`
	`|-> Logging (LOG)`
	`|-> tpLOG_open()`

API to log a fatal error into the system logger.	`Modules`
	`|-> Log And Trace`
	`|-> Logging (LOG)`
	`|-> tpLOG_fatal()`

API to log a non-fatal error into the system logger.	`Modules`
	`|-> Log And Trace`
	`|-> Logging (LOG)`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

API/Type Description	Doxygen Path
	|-> **tpLOG_error()**
API to log a warning into the system logger.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_warn()**
API to log an informative message into the system logger.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_info()**
API to log a debug message into the system logger.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_debug()**
API to log a verbose debug message into the system logger.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_verbose()**
API to log an “Out of Memory” message into the system logger.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_oom()**
API to log an OS error message into the system logger.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_osErr()**
API to close the connection with the system logger.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_close()**
API to set the log level within a specific application.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_setLogLevel()**
API to get the current log level within a specific application.	Modules |-> Log And Trace |-> Logging (LOG) |-> **tpLOG_getLogLevel()**

For an application to start logging messages into the system log, a connection is necessary between it and syslog, specifying the application name and the domain where the messages will be logged. The system then will log only the necessary messages depending on the log level. All APIs in the form `tpLOG_*()` are wrappers of the different log severities defined by syslog.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Note: `tpLOG_verbose()` API will log a "debug" message. `tpLOG_oom()` and `tpLOG_osErr()` requires that log level is at least `E_LOG_SEV_ERROR` to be displayed.

The log level for the specific application can be retrieved with the `tpLOG_getLogLevel()` API and changed with `tpLOG_setLogLevel()` API. The log level is persisted across boots in a very simple way for development environments only.

Note: *Application teams are expected to persist this log level in a secure and reliable manner. Telematics Platform does not guarantee this solution is robust for production environments.* `tpLOG_open()` *will try to read the last persisted value in order to set it correctly.*

How to log application messages into the system logger

The next example shows a very simple application logging different traces depending on severities.

```
#include "tplog.h"
#include <glib.h>
#include <stdlib.h>
#include <stdio.h>

#define APP_NAME "logging-app"
#define BIG_RECOVERY_BUFFER 2048

void dummyCheck(void);
void criticalCheck(void);

int main(int argc, char** argv) {
	tpLOG_open(APP_NAME, TP_LOG_DOMAIN_PRODUCT);

	dummyCheck();
	criticalCheck();

	tpLOG_close();

	return EXIT_SUCCESS;
}

void dummyCheck() {
	tpLOG_verbose("Starting dummy check ...");

	FILE* fp = fopen("/data/unknown", "r");

	if (fp == NULL) {
		tpLOG_osErr("fopen", "/data/unknown");
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
        } else {
                tpLOG_debug("Doing some important actions");
                tpLOG_info("Dummy check success!")
                fclose(fp);
        }
}

void criticalCheck() {
        tpLOG_verbose("Performing critical check on application.");
        guint error_count = 0;

        tpLOG_debug("Creating a buffer to save critical data");
        gchar* buffer = (gchar *) malloc(BIG_RECOVERY_BUFFER);

        if (buffer == NULL) {
                tpLOG_oom();
        }

        // Doing important operations...
        if (error_count > 0) {
                tpLOG_fatal("Critical check failed with %d errors", error_count);
        } else {
                tpLOG_info("Critical check success!");
        }

        free(buffer);
}
```

The result of running a simple program like this is shown below with the different log severities:

E_LOG_SEV_FATAL

E_LOG_SEV_ERROR

```
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
```

E_LOG_SEV_INFO

```
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
local0.info logging-app[7683]: main:Current log level is: E_LOG_SEV_INFO
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
```

E_LOG_SEV_DEBUG

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
local0.info logging-app[7683]: main:Current log level is: E_LOG_SEV_DEBUG
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
local0.debug logging-app[7683]: criticalCheck:Creating a buffer to save critical
data
```

E_LOG_SEV_VERBOSE

```
local0.info logging-app[7683]: main:Current log level is: E_LOG_SEV_VERBOSE
local0.debug logging-app[7683]: dummyCheck:Starting dummy check ...
local0.err logging-app[7683]: dummyCheck:fopen /data/unknown failed: 'No such file
or directory'
local0.debug logging-app[7683]: criticalCheck:Performing critical check on the
application.
Local0.debug logging-app[7683]: criticalCheck:Creating a buffer to save critical
data
```

How to parse application logs

All application logs have the following format:

```
May  3 03:47:14 localhost local0.debug logging-app[991]: main:Goodbye cruel world!
```

Where:

`May 3 03:47:14`	Current timestamp of when the trace was logged, based on the local time.
`Localhost`	Name of the host (this is useful when having remote servers to log information).
`Local0`	Domain to log, in this case: `TP_LOG_DOMAIN_PRODUCT`.
`Debug`	Severity of the message.
`Logging-app`	Name of the application. This name matches the identifier used in `tpLOG_open()`.
`991`	Process ID of the application.
`Main`	Name of the function.
`Goodbye cruel world!`	Actual message logged into the system.

Having this format in mind, it's possible to apply a regex filtering to display only the required amount of logs: filter by application name, domain, severity, etc.

How to parse kernel messages

These message are intermixed with the application messages on the same `messages` file. They provide useful information about drivers and other operations related to the Linux kernel. These are not expected to show in chronological order because of the nature of its operations.

```
[  352.538001] PM: suspend entry 2018-05-16 01:02:16.569412158 UTC
[  352.544162] PM: Syncing filesystems ... done.
[  352.586617] Freezing user space processes ... (elapsed 0.004 seconds) done.
[  352.596534] Freezing remaining freezable tasks ... (elapsed 0.001 seconds) done.
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

The main difference that can be seen is that the timestamp is a positive value in seconds since the kernel start up followed by the specific function and the actual message. `Dmesg` utility can be used to show kernel ring buffer messages into the console.

3.1.3.3 Testing/Error Reporting

Not applicable

3.2 Logging: Configuring and routing log generation

3.2.1 Overview

3.2.1.1 Platform Content Overview

Telematics Platform provides means to configure the way the logs are stored in the file system. These configurations affect the amount of data being written in memory. Application teams should be aware of the constraints on the total amount of memory allocation in their hardware at the moment of designing the solution.

Also, syslog provides a configuration file that can be used to route specific logs into different locations, depending on the severity and the domain they belong. By default, Telematics Platform routes all messages into `/data/log/messages*` with the exception of all fatal logs generated by the Platform and critical kernel logs, which are stored in `/data/log/critical`.

> **Note:** `/var/log` path is a direct link to `/data/log`, so accessing both results in reading/writing the same log files.

Real files is not the only way to store the generated logs, but these can be send to a remote server (as long as this host is reachable) or in RAM, via virtual files.

3.2.1.2 Product Expectations

The following expectations must be met by products (users of the platform) in order for this feature to work as designed:

1) Platform is configured properly (see Extending Platform Functionality section for details).

2) Application logs are routed properly into the desired location inside `syslog.conf` file.

3) Application teams should verify the flash memory has enough space (`file_size` x `rotate_count`) of storage for the generated log files. That applies for all files configured inside a rule. See FAQs section for memory considerations when designing these rules.

3.2.1.3 Extending Platform Functionality

The following describes the platform configuration provided for this feature:

LOG-FILE-SIZE

Configuration Type:	Run-Time (Persistency) Key-Value
Description:	Option to configure the total size in KB of a log file before rotation. Maximum allowed size is 2097152 (2 GB). When this value is 0, only one file is created with no maximum size.
Values:	Integer (size in kilobytes) Platform default: 20480 (20 MB)

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

	Integer '0' – no maximum size configured. This value is **NOT** recommended. All logs will be stored on a single file until no memory is available.
Attributes:	- Product able to override at compile time - Run-time changes NOT take effect immediately (system reboot is required or restart of the busybox-syslog service).

LOG-ROTATE-COUNT

Configuration Type:	Run-Time (Persistency) Key-Value
Description:	Option to configure the maximum count of rotated files.
Values:	Integer value of maximum log files to rotate. Besides the default log file with no extension, this is the maximum number of rotated files with the next format: <log_file>.0, <log_file>.1, <log_file>.2, …, <log_file>.n – 1 For example, for a defined `messages` file and this value being 5 there will be six files: `messages`, `messages.0`, `messages.1`, `messages.2`, `messages.3`, `messages.4` Platform default: 5 Integer '0' – feature disabled (only one log file will be created; every time the file size exceeds, it will be recreated).
Attributes:	- Product able to override at compile time - Run-time changes NOT take effect immediately (system reboot is required or restart of the busybox-syslog service).

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not applicable

3.2.2.2 Setup Instructions

Refer to TP_F101U-00026 for more information on how to update values in Persistence items.

3.2.2.3 Setup Troubleshooting

Not applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

Not applicable

3.2.3.2 Detailed Instructions

For all rules located inside `syslog.conf`, the log daemon will route the proper domains and severities to the specific locations.

How to route messages from domains into different locations

Next example shows how to separate the application development domains to be logged in different files (regular or virtual).

```
# syslogd configuration file
#
# Log all Facilities and all Priorities to messages file
#
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
*.*                                        /data/log/messages
local0.debug                               /dev/shm/debug
local1.*                                   /dev/console
```

The first rule will send all messages to `/data/log/messages`, the standard output log files from Telematics Platform.

The second rule will forward all critical traces from the `TP_LOG_DOMAIN_PRODUCT` domain to a shared memory location in RAM. This is useful when needing to increase the performance and save space disk.

The third rule will forward all traces from `TP_LOG_DOMAIN_PD_FLEX0` domain to the system console over UART.

The resulting logs for these files would look something like below. Notice the combination of messages on the default `messages` file, but a filtered output on the customized location:

`/data/log/messages`

```
[...]
May 16 01:11:28 localhost local0.debug log-test[6828]: dummyCheck:Starting dummy check ...
May 16 01:11:28 localhost local6.debug test: Auto-generated message
May 16 01:11:29 localhost local0.debug log-test[6828]: dummyCheck:Doing some important actions
May 16 01:11:30 localhost local0.warn log-test[6828]: dummyCheck:Dummy check failed! Performing critical check.
May 16 01:11:31 localhost local6.debug test: Auto-generated message
May 16 01:11:32 localhost local0.debug log-test[6828]: criticalCheck:Performing critical check on the application.
May 16 01:11:32 localhost local0.err log-test[6828]: criticalCheck:Trying to save critical data into the filesystem
May 16 01:11:32 localhost local6.debug test: Auto-generated message
May 16 01:11:33 localhost local6.debug test: Auto-generated message
May 16 01:11:33 localhost local0.err log-test[6828]: criticalCheck:46 No memory
May 16 01:11:34 localhost local0.err log-test[6828]: criticalCheck:fwrite No space left to save data. Failed: 'Success'
May 16 01:11:34 localhost local0.emerg log-test[6828]: criticalCheck:No space left on partition.
[...]
```

`/var/shm/debug`

```
May 16 01:11:28 localhost local6.debug test: Auto-generated message
May 16 01:11:31 localhost local6.debug test: Auto-generated message
May 16 01:11:34 localhost local6.debug test: Auto-generated message
May 16 01:11:35 localhost local6.debug test: Auto-generated message
May 16 01:11:36 localhost local6.debug test: Auto-generated message
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
May 16 01:11:36 localhost local6.debug test: Auto-generated message
May 16 01:11:38 localhost local6.debug test: Auto-generated message
```

Important note: A mandatory rule is set on `syslog.conf` file to log fatal logs into `/data/log/critical` and this shouldn't be modified.

```
# (DO NOT CHANGE THIS RULE)
# Always store platform fatal logs and critical
# kernel logs in critical file
# TP_LOG_DOMAIN_PLATFORM = LOG_LOCAL7
#
local7.emerg                        /data/log/critical
kern.crit                           /data/log/critical
```

3.2.3.3 Testing/Error Reporting

Not applicable

4 Core dumps generation customization

4.1 How to configure core dump generation attributes

4.1.1 Overview

4.1.1.1 Platform Content Overview

Telematics platform supports Core dumps generation which provide the ability to debug applications that have unexpected behavior.

Certain build flags detailed in this document allows to configure some attributes to customize the core dump generation as needed.

4.1.1.2 Product Expectations

A way to customize core dumps generation based on project needs. This customization needs to happen before building the image.

4.1.1.3 Extending Platform Functionality

Not applicable

4.1.2 Setup

4.1.2.1 Tools/Equipment Needed

Not applicable

4.1.2.2 Setup Instructions

Not applicable

4.1.2.3 Setup Troubleshooting

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

4.1.3 Usage

4.1.3.1 Basic Instructions

In order to be able to use the customization environment variables, systemd-coredump utility needs to be enabled to generate the core dump files. By default, this utility is selected as part of CAS_CORE_PATTERN default value.

The table below indicates which variables depend on systemd-coredump.

These environment variables can tbe customized as needed in the "board" configuration file:

ENVIRONMENT VARIABLE	DESCRIPTION	DEFAULT VALUE	SYSTEMD DEPENDENT
CAS_CORE_SIZE	Maximum size per core dump file	unlimited	X
CAS_CORE_DIR	Directory where core files will reside	/data/log/cores	
CAS_CORE_PATTERN	Template to name core files	|/lib/systemd/systemd-coredump %p %u %g %s %t %c %e	
CAS_CORE_OVERALL_SIZE	Maximum space to be occupied with core dump files	100M	X

4.1.3.2 Detailed Instructions

All of these environment variables and many others are defined in the "board" file which can be found in `<p1>/project/<project_name>/config/` path.

For `CAS_CORE_SIZE` environment variable, the unlimited default value means that there is no established size limit for a single core file. So, `CAS_CORE_OVERALL_SIZE` environment variable will be used to limit the core file size. If the size of individual core files needs to be limited, the default value can be replaced with a maximum allowed size value in units of megabytes (MB). The syntax for the supported values are the word "unlimited" and the actual size: <*size*>M (e.g., 100M for 100MB, 50M for 50MB, etc). No other syntax is supported (e.g., cannot set size to less than 1 MB, use 1000M for 1 GB size limit).

Example - Set individual core file size limit to 25 MB:

```
export CAS_CORE_SIZE=25M
```

For `CAS_CORE_OVERALL_SIZE` environment variable, the syntax is the same as `CAS_CORE_SIZE` except that "unlimited" value is not allowed. **This value can not exceed the size of the partition where the core files will be stored** (e.g., `/data` for default value of `CAS_CORE_DIR`).

Example - Set maximum size for all core files to 200 MB:

```
export CAS_CORE_OVERALL_SIZE=200M
```

When the sum of the size of all the core files reaches the `CAS_CORE_OVERALL_SIZE` value, the oldest core dumps will be eliminated as needed to allow new ones to be created.

For `CAS_CORE_DIR`, the folder given as parameter will be created with '755' permissions if it doesn't exists.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

For `CAS_CORE_PATTERN`, the default values allows systemd-coredump to handle the core files generation. Even if several parameters are sent to systemd-coredump, only the process or thread comm value (%e) is used and the core file is generated with "core.%e" core pattern. For more information about the core pattern and the possible parameters, please refer to https://man7.org/linux/man-pages/man5/core.5.html page, section "Naming of core dump files".

There is a limitation of only one core file per process. In case that the system needs to generate a new core file of a process that has a core file already generated, the old one will be removed so only the most recent one will persists. This logic is applied before the rotation mechanism.

Once the customization is set and running, the only way to modify these values is by performing a software update to a version that includes the new changes.

Notes:

I. Consider that reducing the core dump size will truncate the file content which could lead to missing information in the core dump files.
II. The `CAS_CORE_SIZE` value can not be greater than `CAS_CORE_SIZE_OVERALL` value.

4.1.3.3 Testing/Error Reporting

Not applicable

5 Known Issues

Not Applicable

6 FAQs

Why am I running out of flash memory?

When designing rules for the syslog.conf file, it's important to note the behaviour caused by the persistence items: All configured files will rotate after the maximum size is reached up to number of configured rotation files. There's no way to configure them separately.

Having many rules means that each independent file will grow over time, using space in the flash memory.

For example, having different files to group logs, with the persistence items set as the default values: `LOG-FILE-SIZE` = 20 MB and `LOG-ROTATE-COUNT` = 5:

```
*.*                                     /data/log/messages
local0.debug                            /data/log/pd/debug
local1.*                                /data/log/pd/messages
```

The space needed for maintaining all rotated files (in different rates, depending on how verbose is an application) would be:

20 MB per file x 6 rotated files x 3 rules = 360 MB of free space on the flash memory.

> **Note:** *Very specific grouping like the second rule may cause problems when analysing issues without the full context of other applications performing tasks in the system.*

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

7 Troubleshooting/Tips and Tricks

Not Applicable

8 References

8.1 Abbreviations/Acronyms/Definitions

Not Applicable

8.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F101U-00023: User Guide Common Library
TP_F101U-00026: User Guide Persistence
TP_F101U-00030 User_Guide_Security

8.3 Website links

https://tools.ietf.org/html/rfc5424
https://linux.die.net/man/5/syslog.conf

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

9 Revision History

Version	Date	Description
01.00	Aug 15, 2019	Initial Release
02.00	Sep 22, 2020	Core dump customization details
03.00	Jul 19, 2021	Remove “Debug Console” section

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted